

Dash Party

Dash Party (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): A retro-arcade party game about race cars, with four game modes (and room to expand) — from classic racing to obstacle courses where two players share control of one car — bringing fast-paced race-car chaos to couch/party play with friends. Named technical features: private-room multiplayer; a dual-to-quad split screen; a server-side solution for a global database and account registration; a mobile controller driven by the phone's gyroscope sensors. Named CS features: mini C++ game loops accepting multiple real-time network connections from mobile devices through the browser; a custom editor to author and import new minigames to grow the mode roster. Design features: 4 game modes, each with its own race courses; a roster of 3 car types; 4 controller layouts (one per mode); 3 unlockable skins per car type (via in-game currency or in-app purchase).

References:

- [The Jackbox Party Pack on Steam](#) (the phone-as-controller party model — players join at a `jackbox.tv` web address and their phones/tablets become the controllers; the precedent for Dash Party's browser-connected gyro controllers)
- [Horizon Chase Turbo — Aquiris](#) (a modern retro-arcade racer explicitly inspired by *Out Run* / *Top Gear* / *Rush*, with a split-screen multiplayer mode — the aesthetic + split-screen precedent)
- [Micro Machines — Wikipedia](#) (top-down miniature-car party racing built around shared-screen local multiplayer — the couch-chaos party-racer precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Dash Party
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Web browser on mobile — the game client runs in the browser (a phone, tablet or a laptop/TV acting as the shared "party screen"), and each player's phone joins as a gyroscope controller through the browser (DeviceOrientation), so no app install is needed; the Native PC build is the development/tools vehicle. **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) hosting the private-room matchmaking/relay and the global account database.

Front end: A retro-arcade 2D renderer with a **dynamic dual-to-quad split screen** (1–4 viewports for 1–4 local racers), each drawing its car, the course, obstacles and HUD; plus the phone-controller web page (a gyroscope steering surface + buttons) and the lobby/room-code screens. **Languages:** Web — HTML/JS/CSS + C++ → WASM (Emscripten, SDL2/3, OpenGL ES); the phone-controller page is HTML/JS reading the DeviceOrientation API.

Back end: The C++ engine core (fixed-timestep race loop, car handling & collision physics, obstacle logic, the 4 mode rulesets, the dual-control resolver, split-screen cameras) + a private-room networking layer that accepts many real-time phone-controller connections through the browser, plus a global-account/leaderboard service. **Languages:** C++ (engine); the relay/matchmaking service on the Linux server.

Backend scope: Local now, server-ready (T2) — private-room relay & global-account DB seam set up & basic-tested on the Linux server by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — it advances every car's handling and collisions, obstacle and course logic, the four mode rulesets, the two-players-one-car control resolver, and the split-screen camera state at a fixed timestep while ingesting live input from the connected phone controllers. The proposal's "mini C++ game loops accepting multiple real-time network connections through the browser" is exactly this requirement.
- *Data-driven:* the 4 game modes and their rules, each mode's courses and obstacle layouts, the 3 car types and their handling stats, the 4 controller layouts, and the skin/currency definitions are hot-reloadable JSON authored via the minigame editor — this is what lets new modes be imported without a recompile.
- *2D:* retro-arcade sprite/tile rendering of cars, courses, obstacles and HUD, drawn into a **dual-to-quad split screen** (dynamic viewport layout for 1–4 local players) — custom-rendered. *3D:* not needed (the arcade look is 2D sprites). *Networking:* private-room multiplayer + many concurrent phone-controller connections (real-time input) + async global leaderboards.
- *Custom tool:* the **minigame editor** — author a new mode's ruleset, course/obstacle layout and controller mapping as data, preview it, and import it to grow the roster beyond the launch four.

- *External DB*: the global account database + per-car skin ownership and leaderboards. *Auth* »: account registration. *Analytics* »: optional — per-mode play counts and drop-off, for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep race loop (C++17/20) that advances car handling, obstacles and the mode ruleset while ingesting live input events relayed from the connected phone controllers; CMake, Git.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; M1G5 physics and M1G11 networking run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component registry in C++ with the 4 modes and their rules, courses & obstacle layouts, the 3 car types and handling stats, the 4 controller layouts and the skin/currency definitions all defined in JSON (nlohmann/json) with hot-reload — the data the minigame editor authors.
- **Prerequisites:** G1 (loop & entity update hooks).
- **Feeds into:** M2G2 (the editor edits this data); M1G5 (car/obstacle data), M1G11 (input mapping).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui debug overlays (per-viewport state, controller latency, physics), Tracy profiling to hold the frame budget with 4 viewports live, Catch2 stress tests (4 cars + many obstacles + 4 concurrent phone controllers), GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the on-device frame budget M2G1 validates.

- **G4 2D Graphics** — the retro-arcade sprite renderer and the **dual-to-quad split screen** are the whole look of the game.

Tech & dependencies

- **Tech: Web build:** Sprite/tile renderer with a dynamic 1-to-4 viewport (split-screen) layout, per-viewport camera, batching, and a retro palette/pixel aesthetic; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType HUD text.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G4 (advanced arcade VFX); every visible frame of the demo.

- **G5 Physics & Collision** — car handling, obstacle collisions and the two-players-one-car control resolution are the mechanical core.

Tech & dependencies

- **Tech:** C++ + GLM math, a custom arcade car model (acceleration, steering, drift), collision/resolution against track bounds, obstacles and other cars, and the dual-control blend for the shared-car mode; broadphase (uniform grid/quadtrees).
- **Prerequisites:** G1 (loop & tick), G2 (car/obstacle data).
- **Feeds into:** The four modes' feel; M2G7 (networked state to sync).

- **G11 Networking Foundation** — private-room multiplayer plus the many real-time phone-controller connections through the browser.

Tech & dependencies

- **Tech: Web build:** WebSockets from the browser client to a relay (Winsock2/BSD sockets or ENet/SDL_net on the server side), a room-code lobby/join protocol, and low-latency ingest of gyroscope/button input from each connected phone; input serialization.
- **Prerequisites:** G1(loop consumes the input), G2 (controller-layout data).
- **Feeds into:** M2G7 (match sync & prediction); the private-room and phone-controller features.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling to hold the frame budget with a 4-way split screen + 4 live controllers, ASan on MSVC, memory pools for cars/obstacles/particles, and Emscripten→WASM validation on the

mobile-web target plus on-device Pixel 9a checks; the Linux server (private-room relay + global account DB) set up (account + software + basic connectivity test) by M2.

- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The web/mobile deliverable and a stable frame budget under full split-screen load.

- **G2 ★ Content Editor — Core** — the **minigame editor**: author a new mode's ruleset, course/obstacle layout and controller mapping as data, preview it, and import it into the roster.

Tech & dependencies

- **Tech:** Dear ImGui editor authoring modes, courses/obstacle placement, car-stat and controller-layout tables as JSON, with round-trip serialization, live preview in the renderer and an undo/redo command stack.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (live preview).
- **Prerequisites:** G2 (mode/course/car data model), M1G4 (rendering for preview).
- **Feeds into:** M2G6 (advanced editor features); the imported-minigames showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the Emscripten web build (game client + phone-controller page) via GitHub Actions.
- **Prerequisites:** G3 (CI and test foundation).
- **Feeds into:** Release/showcase builds of every Dash Party goal.

- **G7 Networking — Match Sync & Prediction** — authoritative race state across the room, with lag compensation for the phone controllers.

Tech & dependencies

- **Tech:** Authoritative server-tick race state, snapshot + delta over the relay, client-side prediction/interpolation for smooth cars, and input lag compensation for the gyroscope controllers.
- **Prerequisites:** M1G11 (networking foundation), M2G1 (optimization).
- **Feeds into:** Smooth private-room multiplayer at showcase; replay from the network stream.

- **G8 Firebase — Accounts, Leaderboards & Analytics** — the server-side global database, account registration and per-mode leaderboards.

Tech & dependencies

- **Tech:** Firebase Auth account registration, Firestore for the global profile / skin-ownership / currency data and per-mode leaderboards, and optional Analytics events, via the REST API from C++ (JS SDK on the web front end).
 - **Prerequisites:** G2 (profile/currency/score data model).
 - **Feeds into:** Account registration + the global leaderboards; skin-ownership persistence.
- **G15 Mobile Platform Port** — the phone-as-controller path and the mobile-web game client: gyroscope input and a touch/frame budget that holds on the phone.

Tech & dependencies

- **Tech: Web build:** Emscripten→WASM game client on mobile browsers, the HTML/JS phone-controller page reading the DeviceOrientation (gyroscope) API and touch buttons, input latency mapping, and frame-budget profiling on the Pixel 9a.
- **Prerequisites:** M2G1 (optimization & platform validation), M1G11 (input transport).
- **Feeds into:** The final mobile deliverable — the gyroscope controller feature.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 1 Art (9) — the design surface is unusually large for a party title (4 distinct game modes each with its own ruleset and courses, 4 controller layouts, the two-players-one-car mechanic, and the skin/currency economy), so two designers split the mode/level/systems design and the UI/UX across the shared-screen HUD and the phone-controller pages; a single artist can carry a cohesive retro-arcade pixel style (3 car types + their skins as palette/variant sets, four modest course/obstacle sets, and the arcade UI) — small, palette-driven sprites keep the volume tractable for one (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the four modes' art volume outgrows one artist, trade the stretch scope, not the headcount. *Scope note (advisory):* four modes + dual-control + split-screen + private-room netcode + phone controllers + editor + server is a very large surface — treat the classic-race mode with 2–4 local players (split screen), the phone-controller transport and the editor as the M1/M2 floor, with the extra modes, online private rooms and the leaderboard server as the stretch.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the party-racer pitch, the four modes and the dual-control USP; primary benchmark e.g. Micro Machines / Horizon Chase Turbo, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving the arcade driving + one mode + the gyro-controller feel is fun) · prototype in Unity
- M1D4 Game Feel & Touch Interaction Specification (the gyroscope steering + the 4 controller layouts, authored as tunable parameters)
- M1D5 Level & Progression Design (the four modes' courses/obstacles and the teach-then-chaos pacing, authored to the editor's data format)
- M1D6 Systems & Economy Design (the 3 car types' handling stats, the skin roster and the in-game-currency/IAP economy — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (car balance across modes + obstacle/pacing tuning via the team's JSON, no code changes)
- M2D4 Game Feel & Feedback Pass ("Juice") — drift/boost/collision feedback, countdown and finish flourishes across the four modes
- M2D6 Progression & Retention Tuning (the skin/currency unlock cadence, upholding the ethics line — cosmetic, never pay-to-win)
- M2D8 Monetization & Scope Integrity Design (the skins-by-currency-or-IAP model held to the module's clean-monetization rule — cosmetics only, no pay-to-win)

- **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the split-screen HUD and the phone-controller page.
- M1U2 UX Flows & Wireframes — Menus → HUD — the room-code join flow (phone), mode/car/skin selection, and the in-race HUD across 1–4 viewports.
- M1U5 Mobile UX Adaptation & Ergonomics — the gyroscope controller designed for thumbs and quick party sessions on the reference device.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — lobby, selection and the phone controller navigable unaided on the mobile target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished HUD + phone controller tested with party-sized groups on the Pixel 9a.
- M2U3 Menu & HUD Motion Design — countdown, position/lap changes and the podium/results sequence animated in the arcade style.

- **Art M1:** 1 artist (cohesive retro pixel style keeps volume tractable)

- M1A1 ★ Art Bible / Proof of Concept (the retro-arcade visual direction — palette, pixel scale, the party-racer tone)
- M1A2 Character Concepts & Model Sheets (the 3 car types as the "cast" + the 3-skins-each variant sheets)

- M1A3 Environment, Props & VFX Concepts (the four modes' courses, obstacles and props)
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality (cars, skins, courses and HUD implemented and consistent in the showcase build)
 - M2A3 Environment Alive — Ambient Animation & Layering (animated crowds, hazards and parallax that make the tracks read as arcade)
 - M2A5 VFX — Critical Effects & Transitions (drift smoke, boost, crashes, pickups and the mode/podium transitions — the arcade juice)
- **TEAM M1:**
 - M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (upbeat chiptune BGM, engine/drift/boost/crash/pickup cues), suitable and correctly triggered.
 - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping race SFX.
- **TEAM M2:**
 - M2TA1 ★ Overall Audio Experience — standard Games floor (chiptune BGM + engine / drift / boost / collision / pickup / countdown / finish SFX, ≥6 types) tuned for four-player party chaos.
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct cues per viewport/car).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the minigame editor), prototype demo, team post-mortem (wk14)